

Math Keygen

Mazzotti's MATH keygen

Author: Mazzotti	Language: C/C++	Upload: 2026-06-26 16:15	Download
Platform: Unix/linux etc.	Difficulty: 3.0	Quality: 2.5	Arch: x86-64
Downloads: 88	Size: 1.80 MB	Writeups: 0	Comments: 0

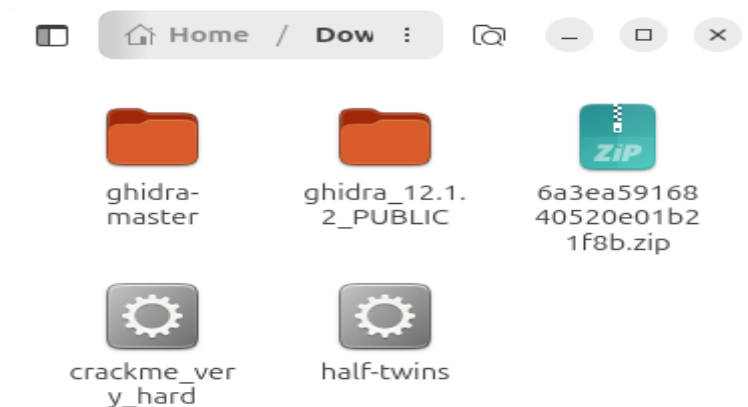
Description

Try to write a keygen (or just find valid password) for this program. Good luck! :3

Comments (0) Writeups (0)

```
adha@Reverse-Engineer:~/Downloads$ unzip 6a3ea5916840520e01b21f8b.zip
Archive: 6a3ea5916840520e01b21f8b.zip
[6a3ea5916840520e01b21f8b.zip] crackme_very_hard password:
password incorrect--reenter:
password incorrect--reenter:
  skipping: crackme_very_hard      incorrect password
adha@Reverse-Engineer:~/Downloads$ unzip 6a3ea5916840520e01b21f8b.zip
Archive: 6a3ea5916840520e01b21f8b.zip
[6a3ea5916840520e01b21f8b.zip] crackme_very_hard password:
  inflating: crackme_very_hard
adha@Reverse-Engineer:~/Downloads$
```

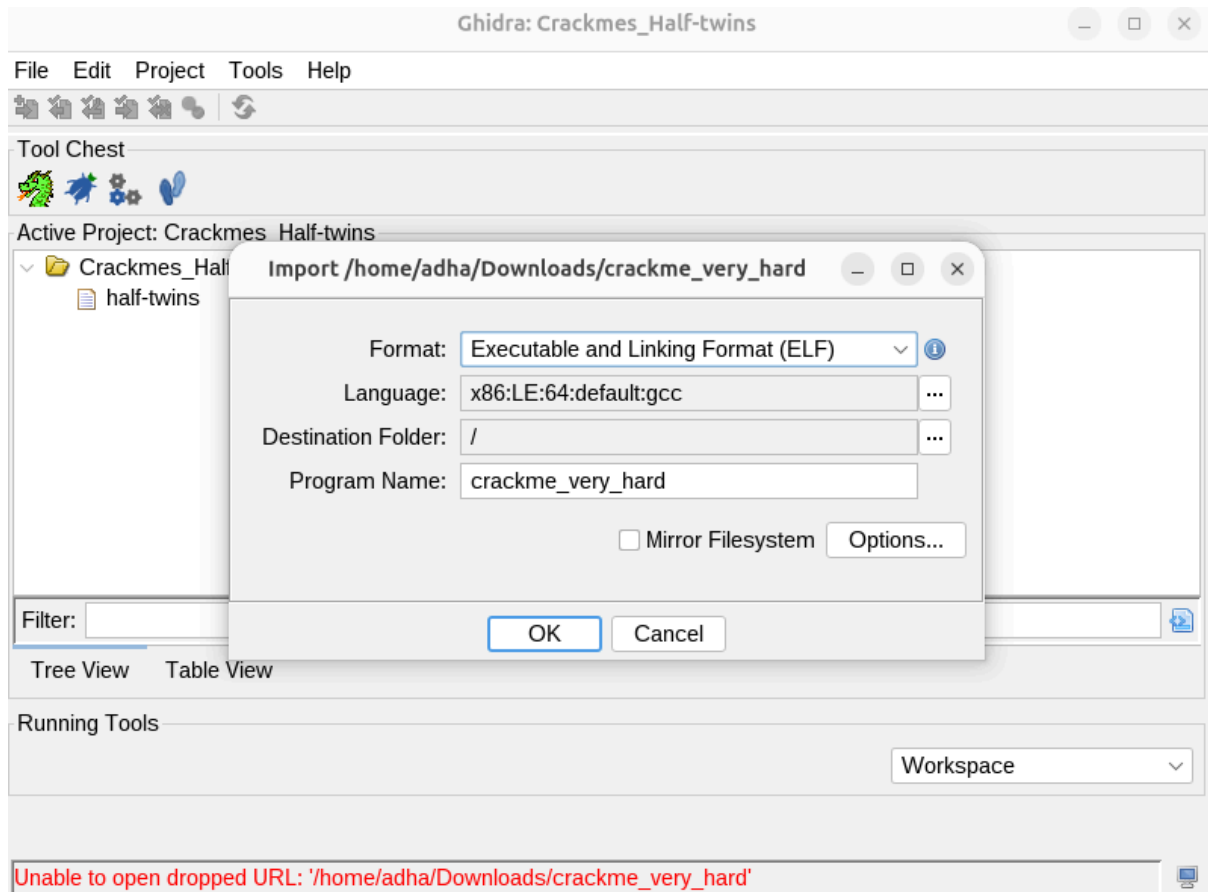
ketika di unzip maka akan ada esan inflating: crackme_very_hard

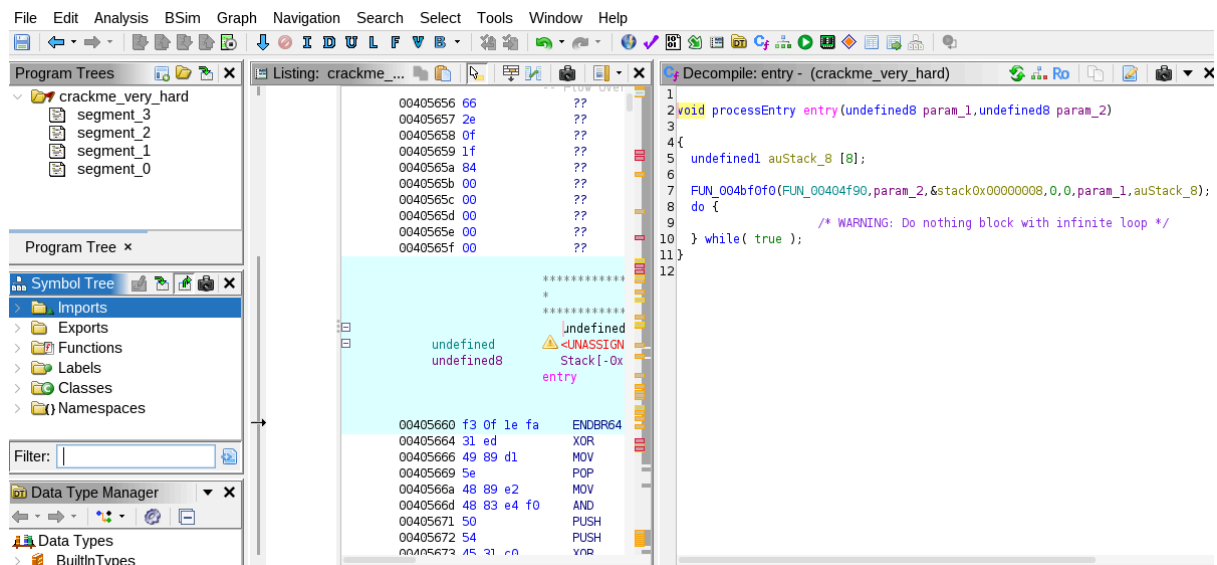


crackme very hard telah terekstrak

```
adha@Reverse-Engineer:~/Downloads$ ./crackme_very_hard
12345678
What is this password? So short / long?
adha@Reverse-Engineer:~/Downloads$ ./crackme_very_hard
adha12
What is this password? So short / long?
```

saat diberikan akses dan dijalankan dengan password biasa. maka crackme_very_hard akan memberikan pesan berupa what is this password? so short / long?





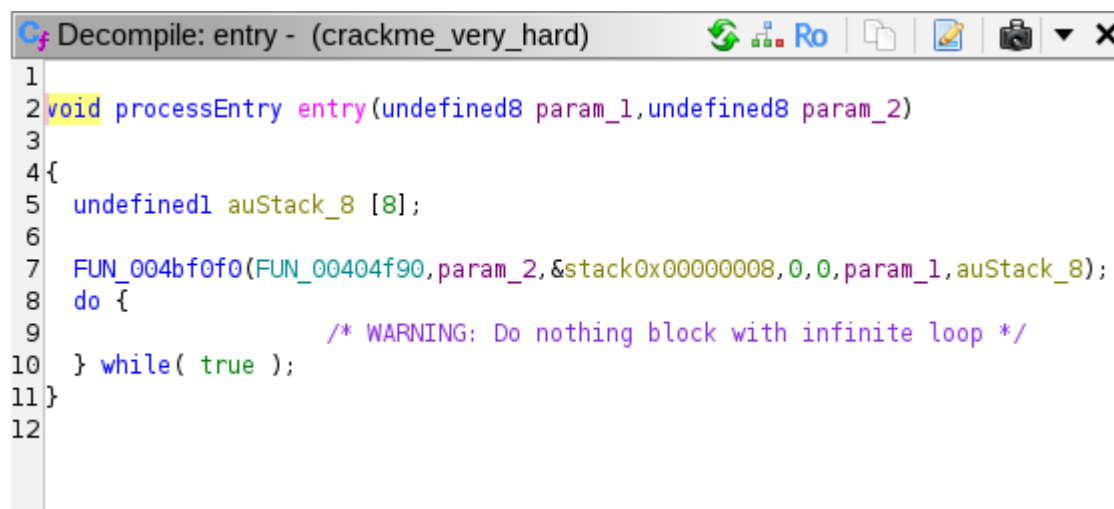
Saat ini kamu sedang berada di fungsi **entry** (atau sering disebut **_start**). Ini adalah titik masuk sistem operasi (Linux) saat pertama kali memuat program ke memori, **bukan** tempat kode yang ditulis oleh pembuat *crackme* dimulai.

Untuk menemukan fungsi main, kamu punya dua cara, yaitu:

cara pertama yang merupakan cara kolom pencarian:

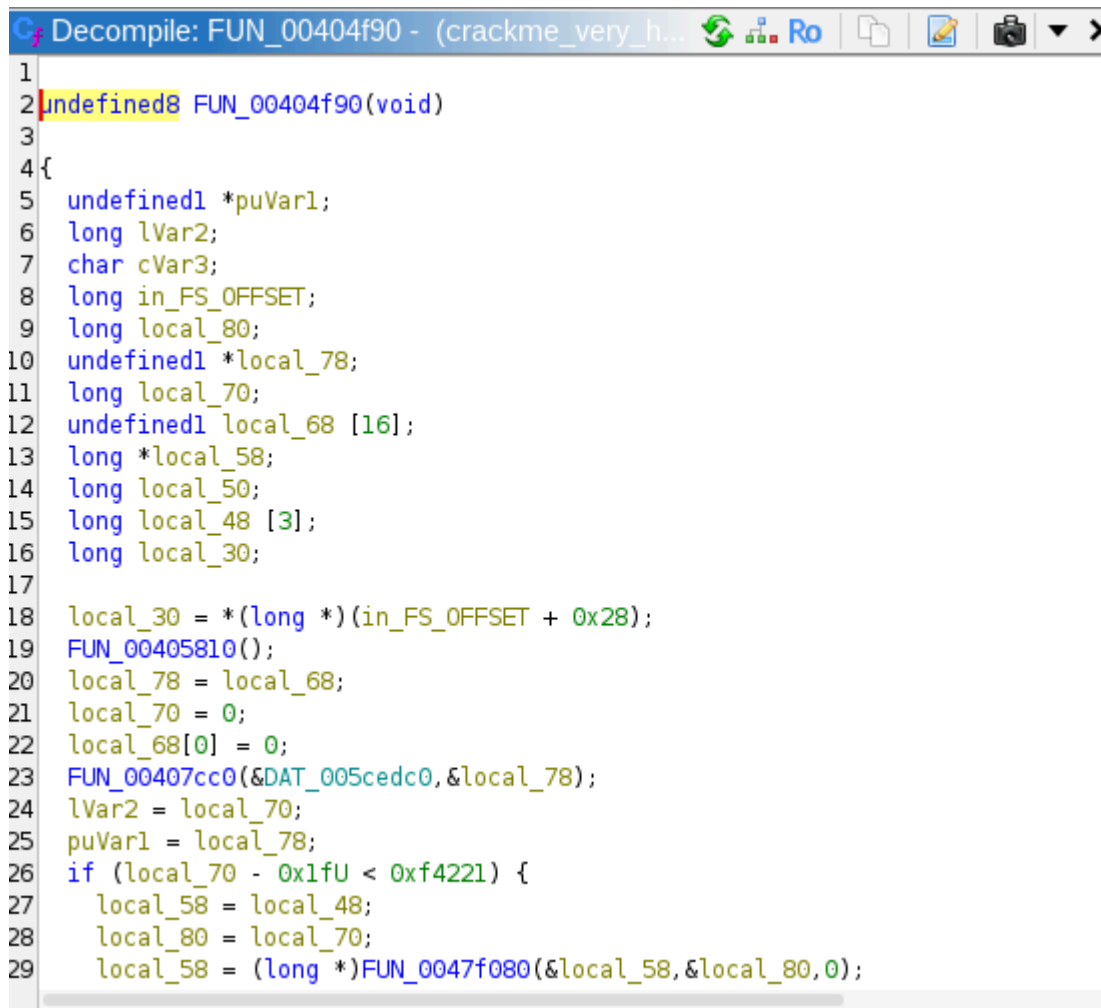
didalam symbol tree kita bisa klik Filter dan langsung mencari nama **main** dalam setiap folder.

tetapi saat tidak menemukan main dan hanya memiliki tampilan kosong artinya pembuat crack me ini tidak ingin semuanya selesai semudah itu. dia ingin membuat kita lebih mencari agar lebih memahami crackme miliknya.



Rahasia Reverse Engineering: Fungsi FUN_004bf0f0 itu sebenarnya adalah fungsi bawaan Linux bernama __libc_start_main. Aturan bakunya, **parameter pertama** yang dimasukkan ke dalam fungsi tersebut adalah **alamat dari fungsi main** yang sebenarnya!

Jadi, fungsi main yang kamu cari bersembunyi di balik alamat **0x404f90**.



```
1
2 undefined8 FUN_00404f90(void)
3
4 {
5     undefined1 *puVar1;
6     long lVar2;
7     char cVar3;
8     long in_FS_OFFSET;
9     long local_80;
10    undefined1 *local_78;
11    long local_70;
12    undefined1 local_68 [16];
13    long *local_58;
14    long local_50;
15    long local_48 [3];
16    long local_30;
17
18    local_30 = *(long *) (in_FS_OFFSET + 0x28);
19    FUN_00405810();
20    local_78 = local_68;
21    local_70 = 0;
22    local_68[0] = 0;
23    FUN_00407cc0(&DAT_005cedc0,&local_78);
24    lVar2 = local_70;
25    puVar1 = local_78;
26    if (local_70 - 0x1fU < 0xf4221) {
27        local_58 = local_48;
28        local_80 = local_70;
29        local_58 = (long *)FUN_0047f080(&local_58,&local_80,0);
```

setelah double klik bagian Fungsi FUN_004bf0f0 maka ghidra akan langsung mengarahkan kita ke dalam Pseudocode decompilernya.

```

C:\Decompile: FUN_00404f90 - (crackme_very_h...
23 FUN_00407cc0(&DAT_005cedc0,&local_78);
24 lVar2 = local_70;
25 puVar1 = local_78;
26 if (local_70 - 0x1fU < 0xf4221) {
27     local_58 = local_48;
28     local_80 = local_70;
29     local_58 = (long *)FUN_0047f080(&local_58,&local_80,0);
30     local_48[0] = local_80;
31     FUN_004011f0(local_58,puVar1,lVar2);
32     local_50 = local_80;
33     *(undefined1 *)((long)local_58 + local_80) = 0;
34     cVar3 = FUN_00405c60(&local_58);
35     FUN_0047f0e0(&local_58);
36     if (cVar3 == '\0') {
37         FUN_004780f0(&DAT_005ceca0,"Get out!",8);
38     }
39     else {
40         FUN_004780f0(&DAT_005ceca0,"OMG! This is very impressive! :3",0x20);
41     }
42 }
43 else {
44     FUN_004780f0(&DAT_005ceca0,"What is this password? So short / long?",0x2
45 }
46 FUN_00405790(&DAT_005ceca0);
47 FUN_0047f0e0(&local_78);
48 if (local_30 == *(long *) (in_FS_OFFSET + 0x28)) {
49     return 0;
50 }
51
/* WARNING: Subroutine does not return */

```

Jika kita geser ke bawah sedikit saja maka kita akan bisa menemukan teks yang sama dimana kita tadi dilarang masuk.

Sekarang coba perhatikan line 26

Jika kondisi ini **salah** (masuk ke blok else di baris 43), program mencetak pesan "So short / long?".

Variabel `local_70` di sini bertindak sebagai penyimpan panjang (*length*) dari *password* kamu.

Ini adalah trik matematika optimasi *compiler* (menggunakan *unsigned integer*) untuk mengecek rentang angka.

- `0x1f` dalam desimal adalah **31**.
- U artinya *Unsigned* (tidak boleh bernilai negatif).
Jika panjang *password* kamu kurang dari 31 (misalnya 10), maka $10 - 31$ seharusnya -21 . Tapi karena ini *unsigned*, hasilnya akan *underflow* menjadi angka positif yang sangat besar (seperti 4.2 miliar), yang mana pasti **lebih besar** dari `0xf4221`. Kondisi pun jadi salah, dan kamu ditendang.

Kesimpulan: Panjang *password* kamu harus **minimal 31 karakter!**

```
adha@Reverse-Engineer:~/Downloads$ ./crackme_very_hard
AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Get out!
```

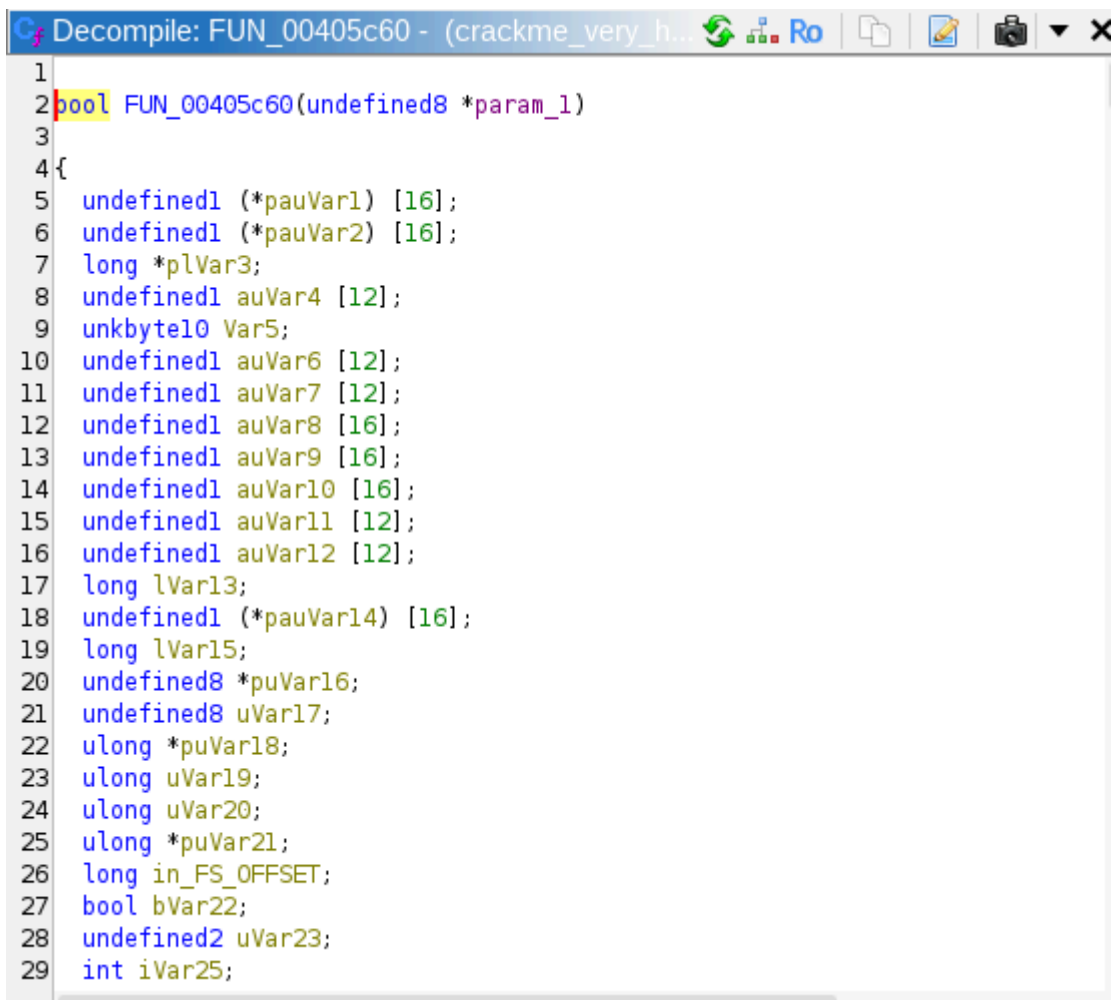
lalu saat kita menggunakan password dengan 31 karakter kita malah disuruh keluar.

kenapa??

itu karena kita hanya tahu kalau karakternya sudah sama. kita masih belum tahu karakter yang benarnya bagaimana.

jadi kuncinya sendiri berada di baris ke 34 dimana tertulis `cvar3` dan memiliki 2 jawaban yaitu "Get Out" dan "OMG!..."

karena itu kita tinggal double klik bagian `FUN_00405c60` dan masuk ke dacompile-nya.



```
Decompile: FUN_00405c60 - (crackme_very_h...
1
2 bool FUN_00405c60(undefined8 *param_1)
3
4 {
5     undefined1 (*pauVar1) [16];
6     undefined1 (*pauVar2) [16];
7     long *plVar3;
8     undefined1 auVar4 [12];
9     unkbyte10 Var5;
10    undefined1 auVar6 [12];
11    undefined1 auVar7 [12];
12    undefined1 auVar8 [16];
13    undefined1 auVar9 [16];
14    undefined1 auVar10 [16];
15    undefined1 auVar11 [12];
16    undefined1 auVar12 [12];
17    long lVar13;
18    undefined1 (*pauVar14) [16];
19    long lVar15;
20    undefined8 *puVar16;
21    undefined8 uVar17;
22    ulong *puVar18;
23    ulong uVar19;
24    ulong uVar20;
25    ulong *puVar21;
26    long in_FS_OFFSET;
27    bool bVar22;
28    undefined2 uVar23;
29    int iVar25;
```

Sekarang kamu berada di dalam fungsi `FUN_00405c60`. Ini adalah "ruang mesin" tempat `crackme` ini melakukan perhitungan matematika (Math Keygen) terhadap *password* 31 karakter yang kamu masukkan.

```

Decompile: FUN_00405c60 - (crackme_very_h...
132 LAB_00405d6d:
133     uVar19 = (long)local_60 - (long)puVar21;
134     if (uVar19 - 1 < 7) {
135 LAB_00405e04:
136         iVar25 = iVar25 + (char)(*pauVar14)[0];
137         if (((pauVar1 != (undefined1 *) [16])(*pauVar14 + 1)) &&
138             (iVar25 = iVar25 + (char)(*pauVar14)[1], pauVar1 != (undefined1
139             ) && (iVar25 = iVar25 + (char)(*pauVar14)[2],
140             pauVar1 != (undefined1 *) [16])(*pauVar14 + 3))) &&
141             (((iVar25 = iVar25 + (char)(*pauVar14)[3], pauVar1 != (undefined1
142             && (iVar25 = iVar25 + (char)(*pauVar14)[4],
143             pauVar1 != (undefined1 *) [16])(*pauVar14 + 5))) &&
144             (iVar25 = iVar25 + (char)(*pauVar14)[5], pauVar1 != (undefined1 (
145             )) {
146             iVar25 = iVar25 + (char)(*pauVar14)[6];
147         }
148         if ((ulong *)0xf < local_60) goto LAB_00405fc0;
149         if (local_60 == (ulong *)0x1) {
150             local_58[0] = (*pauVar2)[0];
151             puVar16 = (undefined8 *) (local_58 + 1);
152             local_68 = (undefined8 *) local_58;
153             goto LAB_00405ea2;
154         }
155     }
156     else {
157         uVar20 = *(ulong *) (*pauVar2 + (long)puVar21);
158         auVar35[0] = (char)uVar20;
159         cVar51 = -(auVar35[0] < '\0');
160         cVar65 = (char)(uVar20 >> 8);

```

```

Decompile: FUN_00405c60 - (crackme_very_h...
456     }
457 }
458 else {
459     uVar19 = 0;
460     uVar20 = 0;
461     lVar13 = 0;
462     bVar22 = false;
463     puVar18 = puVar21;
464     if (puStack_80 != puVar21) {
465         do {
466             bVar22 = CARRY8(uVar20, *puVar18);
467             uVar20 = uVar20 + *puVar18;
468             lVar13 = lVar13 + (ulong)bVar22;
469             uVar19 = uVar19 + 1;
470             puVar18 = puVar18 + 1;
471         } while (uVar19 < (ulong)((long)puStack_80 - (long)puVar21 >> 3));
472         bVar22 = uVar20 == 0x24347a11c16a54a6 && lVar13 == 0x1d;
473     }
474     if (puVar21 != (ulong *)0x0) {
475         thunk_FUN_004f5170(puVar21, local_78 - (long)puVar21);
476     }
477     if (local_40 == *(long *) (in_FS_OFFSET + 0x28)) {
478         return bVar22;
479     }
480 }
481 /* WARNING: Subroutine does not return */
482 FUN_0052cdb0();
483 }
484

```

Nah berdasarkan kode yang sudah kita baca di atas maka kita dapat menarik kesimpulan bahwa:

line pada bariss 472:

- Variabel uVar20 (hasil kalkulasi utama) harus sama persis dengan nilai *Hexadecimal* **0x24347a11c16a54a6**.

- Variabel IVar13 (hasil kalkulasi tambahan/carry) harus bernilai **0x1d** (dalam desimal artinya **29**).

Kesimpulan Logikanya:

Meskipun kodenya terlihat rumit dengan banyak operasi *Shift* (>>), intinya program ini hanya melakukan **Penjumlahan ASCII**. Program mengambil *password* yang kamu ketik, memotongnya menjadi blok-blok memori (biasanya per 8 karakter karena ini arsitektur 64-bit), mengubah setiap karakter menjadi nilai angkanya (ASCII), lalu **menjumlahkan semuanya secara terus-menerus**.

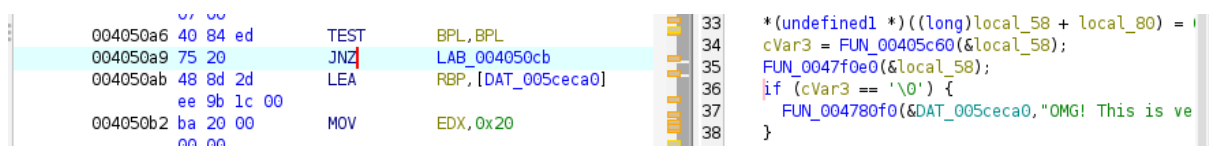
- uVar20 menyimpan total hasil penjumlahan karakternya.
- IVar13 menyimpan nilai *Carry* (sisal/luapan memori jika hasil penjumlahannya melebihi batas tipe data).

Tapi setelah saya dengan lamanya berputar putar di bagian itu, saat saya baca lagi ternyata ada bagian yang:

```
if (2000.0 < (double)(IVar15 - IVar13) / 1000000.0) {
    FUN_004780f0(&DAT_005ceca0, "No stepping bro. Not on this level.", 0x23);
```

yyanng artinya kita tidak bisa menggunakan anti-debugging (sseperti yang saya lakukan selama 2 jam)

kita sekarang hanya perlu pergi ke



yups baris 36 kita bisa melihat

if (cVar3 == '\0') {

kita hanya perlu melakukan binary patching dimana Kita akan membajak alur logikanya sehingga program akan memberikan kemenangan (*OMG!*) meskipun *password* yang kita masukkan salah.

yaituu dengan mengganti **JZ** (*Jump if Zero*) menjadi JNZ atau (Jump Not Zero). tapi disini karena kita menggunakan Ghidra. sekalian saja kita menggunakan NOP (No Operation) Artinya: "Program, jangan lakukan apa-apa di baris ini". Jadi, instruksi JNZ yang tadinya memerintahkan program untuk "lompat jika salah" akan berubah menjadi "diam saja". Program pun akan "jatuh" (tidak lompat) dan terpaksa menjalankan blok kode "*OMG!*" yang kita inginkan.

Karena kamu sudah mengganti JNZ menjadi NOP, program sekarang kehilangan "kemampuan" untuk melompat ke pesan kesalahan "Get out!". Sekarang, apa pun *password* yang kamu masukkan, alur program akan selalu "**jatuh**" (melanjutkan eksekusi) ke baris kode di bawahnya, yang merupakan rute menuju pesan sukses: "**OMG! This is very impressive! :3**".

setelah penggantian kita bisa simpan file dengan cara:
klik file (Dipojokk samping atas) > Export Program > Simpan formatnya sebagai Original File > Pilih nama dan lokasi > berikan akses dan run > masukkan kata sandi dengan 32 karakter > kita berhasil menaklukkan MATH_KEYGEN.

```
adha@Reverse-Engineer:~/Downloads$ ls
6a3ea5916840520e01b21f8b.zip  crackme_very_hard  ghidra-master  half-twins  password_crackme.txt
crackme_hard_solver.py        crackme_very_hard_patched  ghidra_12.1.2_PUBLIC  keygen.py
adha@Reverse-Engineer:~/Downloads$ chmod +x crackme_very_hard_patched
adha@Reverse-Engineer:~/Downloads$ ./crackme_very_hard_patched
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
OMG! This is very impressive! :3
adha@Reverse-Engineer:~/Downloads$
```